



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA
e INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 02

NOMBRE COMPLETO: VELARDE VALENCIA JOSUE

N° de Cuenta: 320342330

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 05/09/2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Para el primer ejercicio, utilicé el código de *CrearLetrasyFiguras()* del main proporcionado por el profesor para crear las figuras de cuadrado café y los triángulos rojo, verde y amarillo, cambiando únicamente los parámetros de color y adecuándolos a los colores requeridos de la siguiente manera:

```
GLfloat vertices_trianguloamarillo[] = {
    //X      Y      Z      R      G      B
    -1.0f,  -1.0f,   0.5f,   1.0f,   1.0f,   0.0f,
    1.0f,   -1.0f,   0.5f,   1.0f,   1.0f,   0.0f,
    0.0f,   1.0f,   0.5f,   1.0f,   1.0f,   0.0f,
};
MeshColor* trianguloamarillo = new MeshColor();
trianguloamarillo->CreateMeshColor(vertices_trianguloamarillo, 18);
meshColorList.push_back(trianguloamarillo);
```

```
GLfloat vertices_triangulorojo[] = {
    //X      Y      Z      R      G      B
    -1.0f,  -1.0f,   0.5f,   1.0f,   0.0f,   0.0f,
    1.0f,   -1.0f,   0.5f,   1.0f,   0.0f,   0.0f,
    0.0f,   1.0f,   0.5f,   1.0f,   0.0f,   0.0f,
};
MeshColor* triangulorojo = new MeshColor();
triangulorojo->CreateMeshColor(vertices_triangulorojo, 18);
meshColorList.push_back(triangulorojo);
```

```
GLfloat vertices_trianguloverde[] = {
    //X      Y      Z      R      G      B
    -1.0f,  -1.0f,   0.5f,   0.0f,   1.0f,   0.0f,
    1.0f,   -1.0f,   0.5f,   0.0f,   1.0f,   0.0f,
    0.0f,   1.0f,   0.5f,   0.0f,   1.0f,   0.0f,
};
MeshColor* trianguloverde = new MeshColor();
trianguloverde->CreateMeshColor(vertices_trianguloverde, 18);
meshColorList.push_back(trianguloverde);
```

```
GLfloat vertices_cuadradocafe[] = {
    //X      Y      Z      R      G      B
    -0.5f,  -0.5f,   0.5f,   0.478f,  0.255f,  0.067f,
    0.5f,   -0.5f,   0.5f,   0.478f,  0.255f,  0.067f,
    0.5f,   0.5f,    0.5f,   0.478f,  0.255f,  0.067f,
    -0.5f,  -0.5f,   0.5f,   0.478f,  0.255f,  0.067f,
    0.5f,   0.5f,    0.5f,   0.478f,  0.255f,  0.067f,
    -0.5f,  0.5f,    0.5f,   0.478f,  0.255f,  0.067f,
};
MeshColor* cuadradocafe = new MeshColor();
cuadradocafe->CreateMeshColor(vertices_cuadradocafe, 36);
meshColorList.push_back(cuadradocafe);
```

Por su parte, el ejercicio 2 consistió en utilizar las figuras creadas anteriormente y realizar nuevas para dibujar una imagen utilizando la proyección ortogonal, utilizando también transformaciones geométricas de rotación y traslación, además, se indicó cambiar el color de las figuras, exceptuando las del anterior ejercicio. De esa forma, tuve que cambiar el color de fondo y agregar 8 figuras nuevas. Para la figura de la izquierda use los 3 triángulos ya hechos y los roté 180 grados en el eje X con la línea `model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));`, y para los cuadrados, modifiqué su escala en el eje X para que quede más pequeña en comparación de la del eje Y y formen un rectángulo delgado, en la línea `model = glm::scale(model, glm::vec3(0.05f, 0.76f, 1.0f));`.

```
//Triangulo Amarillo
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.7f, 0.3f, -3.0f));
model = glm::scale(model, glm::vec3(0.1f, 0.1f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshColorList[0] -> RenderMeshColor();

//Triangulo Rojo
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.7f, 0.1f, -3.0f));
model = glm::scale(model, glm::vec3(0.1f, 0.1f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
meshColorList[1] -> RenderMeshColor();

//Triangulo Verde
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.7f, -0.1f, -3.0f));
model = glm::scale(model, glm::vec3(0.1f, 0.1f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
meshColorList[2] -> RenderMeshColor();

//Cuadrado Cafe 1
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.82f, 0.18f, -3.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.76f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
meshColorList[3] -> RenderMeshColor();

//Cuadrado Cafe 2
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.58f, 0.18f, -3.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.76f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES
meshColorList[3] -> RenderMeshColor();
```

Para la segunda figura creé nuevos triángulos con la forma específica del triángulo inferior izquierdo, pues no fue posible utilizar los ya existentes puesto que la rotación en el eje Z deformaba la figura demasiado. Además de los 3 colores ya usados anteriormente en triángulos, hice uno más color naranja y un cuadrado

color azul que contraste con el cuadrado café (ya existente). La rotación de cada triángulo se modificó según la esquina que debía representar, es por esto que todos tienen un nivel de rotación diferente o no lo tienen. Los cuadrados además de una escala distinta entre sí están rotados sobre el eje X.

```
GLfloat vertices_triangulonaranja2[] = {
    //X      Y      Z      R      G      B
    -1.0f,  -1.0f,   0.5f,   1.0f,   0.5f,   0.0f,
    -1.0f,   1.0f,   0.5f,   1.0f,   0.5f,   0.0f,
    1.0f,   -1.0f,   0.5f,   1.0f,   0.5f,   0.0f,
};
MeshColor* triangulonaranja2 = new MeshColor();
triangulonaranja2->CreateMeshColor(vertices_triangulonaranja2, 18);
meshColorList.push_back(triangulonaranja2);

GLfloat vertices_trianguloverde2[] = {
    //X      Y      Z      R      G      B
    -1.0f,  -1.0f,   0.5f,   0.0f,   1.0f,   0.0f,
    -1.0f,   1.0f,   0.5f,   0.0f,   1.0f,   0.0f,
    1.0f,   -1.0f,   0.5f,   0.0f,   1.0f,   0.0f,
};
MeshColor* trianguloverde2 = new MeshColor();
trianguloverde2->CreateMeshColor(vertices_trianguloverde2, 18);
meshColorList.push_back(trianguloverde2);

GLfloat vertices_trianguloamarillo2[] = {
    //X      Y      Z      R      G      B
    -1.0f,  -1.0f,   0.5f,   1.0f,   1.0f,   0.0f,
    -1.0f,   1.0f,   0.5f,   1.0f,   1.0f,   0.0f,
    1.0f,   -1.0f,   0.5f,   1.0f,   1.0f,   0.0f,
};
MeshColor* trianguloamarillo2 = new MeshColor();
trianguloamarillo2->CreateMeshColor(vertices_trianguloamarillo2, 18);
meshColorList.push_back(trianguloamarillo2);

GLfloat vertices_triangulorojo2[] = {
    //X      Y      Z      R      G      B
    -1.0f,  -1.0f,   0.5f,   1.0f,   0.0f,   0.0f,
    -1.0f,   1.0f,   0.5f,   1.0f,   0.0f,   0.0f,
    1.0f,   -1.0f,   0.5f,   1.0f,   0.0f,   0.0f,
};
MeshColor* triangulorojo2 = new MeshColor();
triangulorojo2->CreateMeshColor(vertices_triangulorojo2, 18);
meshColorList.push_back(triangulorojo2);
```

```
GLfloat vertices_cuadradoazul[] = {
    //X      Y      Z      R      G      B
    -0.5f,  -0.5f,   0.5f,   0.0f,   0.8f,   0.8f,
    0.5f,   -0.5f,   0.5f,   0.0f,   0.8f,   0.8f,
    0.5f,   0.5f,   0.5f,   0.0f,   0.8f,   0.8f,
    -0.5f,  -0.5f,   0.5f,   0.0f,   0.8f,   0.8f,
    0.5f,   0.5f,   0.5f,   0.0f,   0.8f,   0.8f,
    -0.5f,  0.5f,   0.5f,   0.0f,   0.8f,   0.8f,
};
MeshColor* cuadradoazul = new MeshColor();
cuadradoazul->CreateMeshColor(vertices_cuadradoazul, 36);
meshColorList.push_back(cuadradoazul);
```

```

//Triangulo Naranja Fig 2
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.25f, -0.05f, -3.0f));
model = glm::scale(model, glm::vec3(0.15f, 0.15f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));//FALSE ES
meshColorList[6]->RenderMeshColor();

//Triangulo Verde Fig 2
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.05f, -0.05f, -3.0f));
model = glm::scale(model, glm::vec3(0.15f, 0.15f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));//FALSE ES
meshColorList[7]->RenderMeshColor();

//Triangulo Amarillo Fig 2
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.25f, 0.25f, -3.0f));
model = glm::scale(model, glm::vec3(0.15f, 0.15f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));//FALSE ES
meshColorList[8]->RenderMeshColor();

//Triangulo Rojo Fig 2
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.05f, 0.25f, -3.0f));
model = glm::scale(model, glm::vec3(0.15f, 0.15f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));//FALSE ES
meshColorList[9]->RenderMeshColor();

//Cuadrado Azul Fig 2
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.10f, 0.10f, -3.0f));
model = glm::scale(model, glm::vec3(0.425f, 0.425f, 1.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));//FALS
meshColorList[10]->RenderMeshColor();

//Cuadrado Cafe Fig 2
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.10f, 0.10f, -3.0f));
model = glm::scale(model, glm::vec3(0.2f, 0.2f, 4.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));//FALS
meshColorList[3]->RenderMeshColor();

```

Para la tercer figura usé nuevamente los triángulos realizados en el primer ejercicio y agregué un triángulo naranja extra, todos fueron acomodados sin modificar su rotación a excepción del triangulo amarillo que fue rotado en 180 grados.

```

GLfloat vertices_triangulonaranja[] = {
    //X      Y      Z      R      G      B
    -1.0f,  -1.0f,   0.5f,   1.0f,   0.5f,   0.0f,
    1.0f,   -1.0f,   0.5f,   1.0f,   0.5f,   0.0f,
    0.0f,   1.0f,   0.5f,   1.0f,   0.5f,   0.0f,
};
MeshColor* triangulonaranja = new MeshColor();
triangulonaranja->CreateMeshColor(vertices_triangulonaranja, 18);
meshColorList.push_back(triangulonaranja);

```

```

//Triangulo Amarillo Fig 3
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.6f, -0.05f, -3.0f));
model = glm::scale(model, glm::vec3(0.15f, 0.15f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshColorList[0]->RenderMeshColor();

```

```

//Triangulo Rojo Fig 3
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.75f, -0.05f, -3.0f));
model = glm::scale(model, glm::vec3(0.15f, 0.15f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE
meshColorList[1]->RenderMeshColor();

```

```

//Triangulo Verde Fig 3
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.45f, -0.05f, -3.0f));
model = glm::scale(model, glm::vec3(0.15f, 0.15f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE
meshColorList[2]->RenderMeshColor();

```

```

//Triangulo Azul Fig 3
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.6f, 0.25f, -3.0f));
model = glm::scale(model, glm::vec3(0.15f, 0.15f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE
meshColorList[5]->RenderMeshColor();

```

```

//Cuadrado Azul Fig 2
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.10f, 0.10f, -3.0f));
model = glm::scale(model, glm::vec3(0.425f, 0.425f, 1.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE
meshColorList[10]->RenderMeshColor();

```

```

//Cuadrado Cafe Fig 2
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.10f, 0.10f, -3.0f));
model = glm::scale(model, glm::vec3(0.2f, 0.2f, 4.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE
meshColorList[3]->RenderMeshColor();

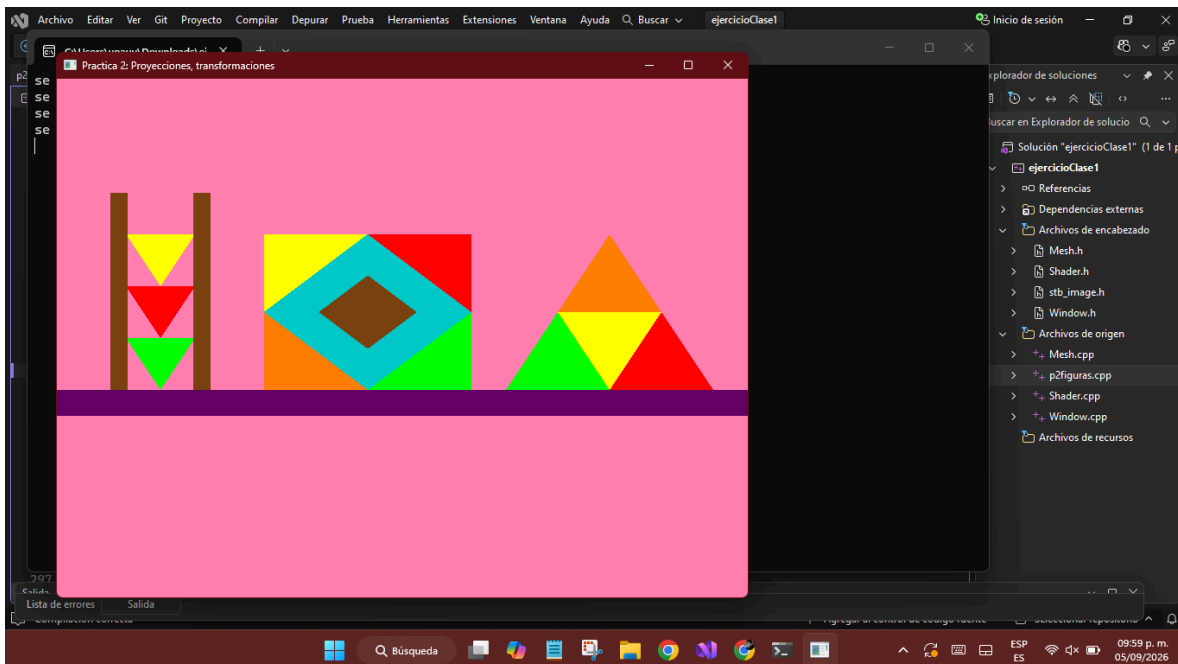
```

Por último, creé una nueva figura para el cuadrado de base con los mismos parámetros que las anteriores, pero cambiándolo a un color morado. Al colocarla en el plano cambié su escala en el eje Y para que quede más pequeña que la del eje X y se vea como un rectángulo horizontal.

```
GLfloat vertices_cuadradoromorado[] = {
    //X      Y      Z      R      G      B
    -0.5f,  -0.5f,   0.5f,   0.4f,   0.0f,   0.4f,
    0.5f,   -0.5f,   0.5f,   0.4f,   0.0f,   0.4f,
    0.5f,    0.5f,   0.5f,   0.4f,   0.0f,   0.4f,
    -0.5f,  -0.5f,   0.5f,   0.4f,   0.0f,   0.4f,
    0.5f,    0.5f,   0.5f,   0.4f,   0.0f,   0.4f,
    -0.5f,    0.5f,   0.5f,   0.4f,   0.0f,   0.4f,
};
MeshColor* cuadradoromorado = new MeshColor();
cuadradoromorado->CreateMeshColor(vertices_cuadradoromorado, 36);
meshColorList.push_back(cuadradoromorado);
```

```
//Cuadrado Morado
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-1.0f, -0.25f, -3.0f));
model = glm::scale(model, glm::vec3(4.0f, 0.1f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[4]->RenderMeshColor();
```

A continuación se presenta el resultado final:



2. Problemas presentados.

No surgieron problemas en la ejecución, tuve más complicaciones en la comprensión de los requerimientos, pues no me quedó claro si se requería cambiar de color todas las figuras o solo hacer la excepción con las que no fueran rojo, amarillo, verde y café, además de lo mencionado para la segunda figura que no pude utilizar los triángulos ya creados porque se deformaban demasiado al rotarlas en Z.

3. Conclusión:

Gracias al desarrollo de la práctica pude comprender cómo realizar funciones que contienen los vértices y colores de figuras, me parece algo muy útil pues solo hace falta llamar a esas funciones para colocarlas y no es necesario escribirlas completas, además de que nos ofrece la opción de modificar su escala, posición y rotación sin cambiar los valores base. La explicación fue bastante práctica y entendible aunque creo que en la clase se enfocó más en las funciones de las figuras 3D que en las 2D.